
Del diseño lógico al diseño físico

Implementación de una base de datos - Parte DML

BASES DE DATOS I - FaCENA | UNNE



Sentencias DML

Lenguaje de Manipulación de Datos (Data Manipulation Language)
INSERT - UPDATE - DELETE

SQL - INSERT



La sentencia **INSERT** es la operación encargada de agregar nuevos registros o filas a las tablas de una base de datos relacional.

Formas de inserción de registros:

- Inserción directa: **INSERT INTO ... VALUES**
- Inserción basada en consultas: **INSERT INTO ... SELECT**

```
-- Inserción directa de registros nuevos a una tabla
INSERT INTO PROVINCIA (cod_provincia,nombre) VALUES
(1, 'Corrientes'), (2, 'Chaco');
```

```
-- Inserción basada en consultas:
INSERT INTO CIUDAD (cod_ciudad, nombre, cod_provincia)
SELECT ciudad_ident, nombre_ciudad, cod_provincia
FROM LOCALIDADES_TEMP WHERE cod_provincia = 2;
```

SQL - INSERT



Durante la ejecución de un **INSERT**, el motor del sistema gestor valida que los datos entrantes respeten las **restricciones de integridad definidas** en el esquema de la tabla:

- **Claves Primarias (PRIMARY KEY) y Unicidad (UNIQUE)**: Impide el ingreso de registros con claves o valores duplicados
- **Integridad Referencial (FOREIGN KEY)**: Comprueba que los valores de las claves foráneas existan en la tabla referenciada
- **Nulidad y Valores por Defecto (NOT NULL / DEFAULT)**: Requiere un valor obligatorio si la columna no admite nulos (NOT NULL), a menos que se haya definido un valor predeterminado (DEFAULT)
- **Reglas de Dominio (CHECK)**: Evalúa condiciones lógicas sobre los valores (como verificar límites numéricos) antes de confirmar la inserción

SQL - UPDATE



La sentencia **UPDATE** es la operación encargada de modificar los valores almacenados en filas o registros existentes dentro de las tablas.

```
-- Actualiza los registros de la tabla que coincidan con la condición (WHERE)
UPDATE PROVINCIA
SET nombre = 'Misiones'
WHERE nombre = 'Pcia. Mis.';
```

- **Cláusula SET** : Define las columnas que recibirán modificaciones y asigna sus nuevos valores, los cuales pueden ser literales explícitos o el resultado de expresiones aritméticas (*descuento = precio * 0.20*)
- **Cláusula WHERE** : Aplica un filtro condicional que determina exactamente qué registros deben actualizarse. Si la cláusula **WHERE** se omite por completo, el motor actualizará el valor de la columna especificada en absolutamente todas las filas de la tabla.

SQL - UPDATE



Al procesar una instrucción **UPDATE**, el motor del sistema gestor valida que los nuevos datos satisfagan las reglas de integridad declaradas en la tabla:

- **Tipos de datos y dominios:** Los valores asignados deben coincidir con el tipo de dato de la columna (int, decimal, varchar, etc.), respetando la precisión, la escala y los límites de almacenamiento asignados.
- **Control de Nulidad (NOT NULL):** Se rechaza la asignación de un valor **NULL** a columnas declaradas como obligatorias.
- **Integridad de Claves (PRIMARY KEY / UNIQUE):** Al modificar valores en columnas clave, el gestor verifica que no se introduzcan valores duplicados dentro de la tabla.
- **Relaciones Referenciales y Reglas (FOREIGN KEY / CHECK):** Comprueba que los cambios respeten las relaciones de claves foráneas con tablas vinculadas y cumplan con las expresiones lógicas evaluadas en las restricciones **CHECK**.

SQL - DELETE

La sentencia **DELETE** es la operación responsable de remover o eliminar registros existentes almacenados en las tablas.

- Borra los registros de la tabla que coincidan con la condición (WHERE)

```
DELETE FROM CIUDAD  
WHERE cod_provincia = 2;
```

- **Cláusula WHERE** : Aplica un filtro condicional que determina exactamente qué registros deben borrarse. Si la cláusula **WHERE** se omite por completo, el motor **BORRARÁ TODAS** todas las filas de la tabla.
- Usar **SIEMPRE DELETE** con **WHERE**.

SQL - DELETE



Durante la ejecución de un **DELETE**, el motor evalúa las reglas de integridad semántica declaradas en el esquema:

- **Integridad Referencial (FOREIGN KEY)**: Si la tabla origen se encuentra vinculada a otras tablas mediante claves foráneas, el motor aplicará las reglas de eliminación configuradas en la DDL (como **ON DELETE RESTRICT**), impidiendo la eliminación de registros si existen tablas hijas que dependen de ese valor.

SQL - DELETE y TRUNCATE



Criterio	DELETE	TRUNCATE
Clasificación de sublenguaje	DML	DDL
Uso de la cláusula WHERE	Permite aplicar filtros condicionales para eliminar filas específicas.	No admite la cláusula WHERE; vacía siempre la tabla en su totalidad.
Mecanismo de borrado	Remueve registros individuales mediante el procesamiento de filas.	Remueve todos los datos de forma masiva afectando la estructura del objeto.
Gestión transaccional y bloqueos	Ejecuta operaciones sobre datos sin commit implícito.	Adquiere bloqueos exclusivos sobre la estructura antes de ejecutarse.

SQL - WHERE en DML

La cláusula **WHERE** cumple la función fundamental de actuar como un **filtro condicional o predicado de selección**.

```
WHERE cod_provincia = 2 AND nombre LIKE 'Cor%';
```

Su objetivo es acotar el alcance de las operaciones de manipulación de datos para que afecten únicamente a las filas que cumplen con los criterios establecidos.

- En una operación `UPDATE`, la cláusula `WHERE` determina qué registros existentes en la tabla deben modificar sus valores.
- En una operación `DELETE`, la cláusula `WHERE` define la condición lógica para la eliminación selectiva de registros.

SQL - WHERE en DML

Para construir un **predicado de selección** (la condición que filtra las filas) en la cláusula WHERE, combinamos **operadores de comparación** (que evalúan expresiones individuales) y **operadores lógicos** (que conectan o modifican esas evaluaciones).

Operadores de comparación

Operador	Significado	Propósito	Ejemplo de Uso en WHERE
=	Igual a	Filtra coincidencia exacta de valores.	WHERE Estado = 'Activo'
<> o !=	No igual a (Diferente)	Filtra filas que no coincidan con el valor.	WHERE Pais <> 'Argentina'
>	Mayor que	Filtra valores estrictamente mayores.	WHERE Precio > 1500.00
<	Menor que	Filtra valores estrictamente menores.	WHERE Stock < 10
>=	Mayor o igual que	Filtra valores mayores o exactamente iguales.	WHERE Edad >= 18
<=	Menor o igual que	Filtra valores menores o exactamente iguales.	WHERE FechaEntrega <= '2026-12-31'
IN	En una lista	Compara un valor con una lista de opciones explícitas o una subconsulta.	WHERE CategoricalID IN (1, 5, 8)
BETWEEN	Entre un rango	Filtra un rango de valores (incluye los límites inicial y final).	WHERE Salario BETWEEN 2000 AND 5000
LIKE	Coincidencia de patrón	Busca patrones de texto usando comodines (% para texto, _ para un carácter).	WHERE Nombre LIKE 'J%' (Inicia con J)
IS NULL	Es nulo	Verifica si el campo carece de valor (un valor vacío o desconocido).	WHERE Telefono IS NULL

SQL - WHERE en DML

Para construir un **predicado de selección** (la condición que filtra las filas) en la cláusula WHERE, combinamos **operadores de comparación** (que evalúan expresiones individuales) y **operadores lógicos** (que conectan o modifican esas evaluaciones).

Operadores lógicos

Operador	Propósito	Regla de Evaluación	Ejemplo de Uso en WHERE
AND	Conjunción lógica	Retorna TRUE solo si todas las condiciones separadas por él son verdaderas.	WHERE Rol = 'Admin' AND Activo = 1 <i>(Debe cumplir ambas cosas)</i>
OR	Disyunción lógica	Retorna TRUE si al menos una de las condiciones es verdadera.	WHERE Ciudad = 'Córdoba' OR Ciudad = 'Rosario' <i>(Cumple con cualquiera de las dos)</i>
NOT	Negación lógica	Invierte el resultado de la condición que le sigue (TRUE pasa a FALSE y viceversa).	WHERE NOT CategoricalID = 3 <i>(También puede usarse como IS NOT NULL o NOT IN)</i>

SQL - WHERE en DML

Orden de Precedencia (Importante)

Cuando se combinan múltiples operadores en una sola cláusula WHERE, SQL los evalúa en un orden estricto de prioridad:

1. Operadores de comparación (=, >, LIKE, etc.)
2. NOT
3. AND
4. OR

Si se necesita alterar este orden natural para evitar errores de lógica, se debe utilizar paréntesis ().

```
- Busca registros con población > 50.000 y que sean de Corrientes, o cualquier registro de Córdoba  
WHERE poblacion_ciudad > 50.000 AND nombre = 'Corrientes' OR nombre = 'Córdoba';
```

```
- Busca registros sean de Corrientes o de Córdoba, pero obligatoriamente tengan una población superior a  
50.000  
WHERE poblacion_ciudad > 50.000 AND (nombre = 'Corrientes' OR nombre = 'Córdoba');
```

SQL - NULL en DML



NULL se utiliza para representar que el valor de un atributo es **desconocido** o no está presente. No hay dos valores **NULL** que sean iguales entre sí, esto afecta a las operaciones DML que se realicen sobre los datos.

La evaluación o comparación entre dos valores **NULL**, o entre un valor **NULL** y cualquier otro dato (incluso dentro de cláusulas de filtrado en sentencias **SELECT**, **UPDATE** o **DELETE**), produce invariablemente un **resultado desconocido** (*unknown*).

Al inspeccionar los resultados de una consulta DML dentro de SQL Server Management Studio (SSMS), los valores nulos se muestran explícitamente etiquetados como (**null**) en la grilla de datos.

- **En sentencias INSERT** : Para ingresar registros con valores nulos, la columna de destino debe haber sido definida en la DDL permitiendo nulos (**NULL**)
- **En sentencias UPDATE** : Permite actualizar el contenido de una columna asignándole el valor **NULL**, siempre que la columna no esté sujeta a restricciones de no nulidad (**NOT NULL**)
- **En sentencias DELETE** : Afecta la selección de las filas a borrar dependiendo de cómo evalúen las condiciones sobre columnas que contienen nulos en la cláusula **WHERE**

SQL - NULL en DML

IS NULL / IS NOT NULL son operadores de comparación que se utilizan para evaluar y filtrar filas en la cláusula `WHERE`

```
WHERE cod_postal IS NOT NULL;
```

Evaluar una comparación directa como `WHERE columna = NULL` o `WHERE columna <> NULL` no devuelve ningún registro, ya que la evaluación resulta en **desconocido** (*UNKNOWN*)

ISNULL() es una **función escalar de reemplazo** propia de Transact-SQL (T-SQL) diseñada para sustituir valores desconocidos dentro de expresiones o selecciones.

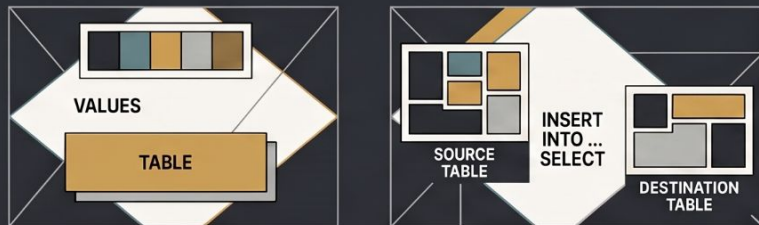
```
WHERE ISNULL(cod_postal, 0) = 0;
```

Lenguaje de Manipulación de Datos (DML) en SQL Server

El Lenguaje de Manipulación de Datos (DML) agrupa las instrucciones responsables de gestionar el ciclo de vida de los registros en tablas relacionales. Su ejecución debe cumplir estrictamente con las restricciones de dominio e integridad definidas en el esquema de la base de datos para garantizar la consistencia de la información.

OPERACIONES DE MANIPULACIÓN DE DATOS (DML)

• INSERCIÓN DE REGISTROS (INSERT)



Permite agregar datos mediante VALUES o desde otra tabla utilizando la estructura INSERT INTO SELECT.

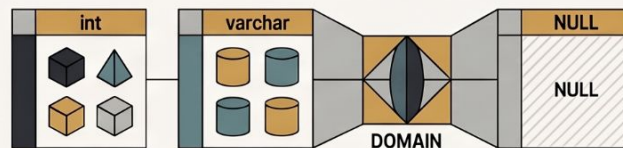
• ACTUALIZACIÓN Y BORRADO CONDICIONAL



Las instrucciones UPDATE y DELETE requieren el uso de la cláusula WHERE para aplicar modificaciones selectivas sobre registros específicos.

INTEGRIDAD DE DATOS Y RESTRICCIONES

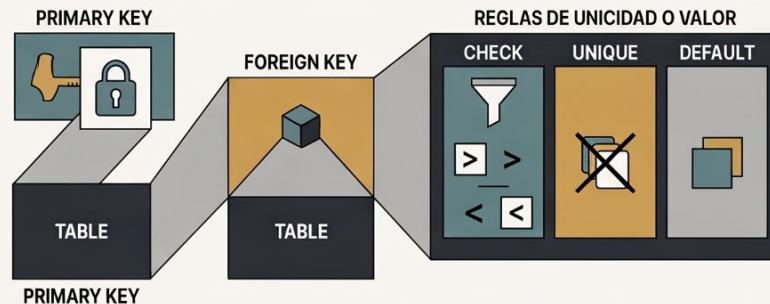
• DOMINIO DE COLUMNA Y VALORES NULOS



Los tipos de datos (ej. int, varchar) definen el dominio de almacenamiento;

el valor NULL indica formalmente un dato desconocido.

• RESTRICCIONES DE INTEGRIDAD RELACIONAL



Las operaciones DML están limitadas por validaciones de clave (PRIMARY KEY, FOREIGN KEY) y reglas de unicidad o valor (CHECK, UNIQUE, DEFAULT).



Bibliografía

- Documentación del Aula Virtual